Filtering and Faceting

filters say yes or no; aggregations count

Watch the 4-min video

In one line: put every non-scoring condition in **filter context** so it skips scoring and can be **cached as a bit set**; compute facet counts with **aggregations** over the current matches, read from columnar **doc values**; and use a **post filter** so a selected facet doesn't hide its own alternatives.

1 The shopping sidebar

A shopper searches **laptops** and gets 12,000 results. The sidebar shows Brand (Lenovo 240, Dell 310...), Price (under $1,000: 3,000), Screen size and Rating, each with live counts that change as the shopper clicks. On a busy site that is thousands of searches a second, and every one needs both the top 20 results and every count, in tens of milliseconds. That works because engines treat filters and counts very differently from text relevance.

2 Queries score, filters don't

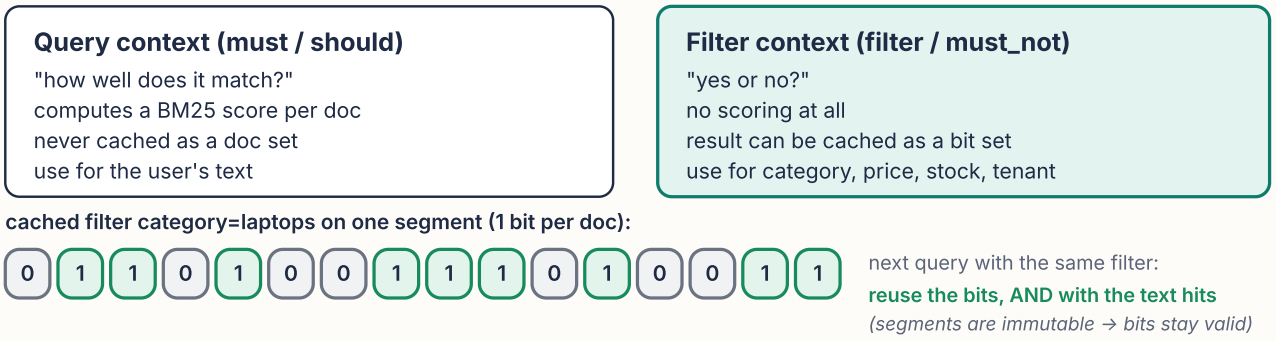


Figure 1. In a bool query, must/should clauses score; filter/must_not clauses only include or exclude, so their results can be cached.

A `bool` query has four clause types: `must` (match and score), `should` (optional, adds score), `filter` (must match, no score) and `must_not` (must not match, no score). Brand, price range, stock, category, tenant ID and permissions are yes/no questions, so they belong in `filter`. That skips BM25 work and lets the engine cache the matching documents.

How the cache works. Elasticsearch's node query cache stores, per segment, a bit set of documents matching a filter. Because segments are immutable (lesson 15), the bits stay valid until the segment is merged away. It is LRU, sized at 10% of heap by default, and Lucene only caches filters that are used repeatedly and only on larger segments (10,000+ docs), since tiny segments are cheap anyway. A separate **shard request cache** stores whole results of `size: 0` requests (aggregation-only, like a facet refresh) and is invalidated whenever the shard refreshes.

Cache-friendly filters. `now` changes every millisecond, so `range: {created: {gte: "now-1h"}}` is never the same filter twice; rounding to `now-1h/m` makes it reusable for a minute.

3 Facet counts are aggregations

```
GET products/_search
{ "size": 20,
  "query": { "bool": {
      "must":   [ { "match": { "name": "laptop" } } ],
      "filter": [ { "term": { "category": "laptops" } },
                  { "term": { "in_stock": true } } ] } },
  "aggs": {
    "brands": { "terms": { "field": "brand", "size": 10 } },
    "prices": { "range": { "field": "price", "ranges": [
        { "to": 500 }, { "from": 500, "to": 1000 }, { "from": 1000 } ] } } } }
```

Figure 2. One request: scored text query, cached filters, a terms facet and a range facet. Range 'from' is inclusive and 'to' exclusive.

- **terms**: groups matching docs by a keyword field and returns the top N buckets with counts (brand, colour, seller).
- **range** / **histogram**: counts per numeric bucket (price bands, screen sizes); **date_histogram** for time.
- **filter** / **filters**: counts for arbitrary conditions ('rating ≥ 4', 'free delivery').
- **Metrics**: min/max price for a slider, avg rating, and **cardinality** (distinct count, approximated with HyperLogLog++).

Aggregations run over exactly the documents that match the query and filters, so counts always reflect the current search. On many shards, `terms` counts can be approximate: each shard returns only its own top `shard_size` buckets (default size × 1.5 + 10), so a brand that is 11th on every shard can be under-counted. The response reports `doc_count_error_upper_bound`; raise `shard_size` if exact top-N matters. For a sidebar, approximate is usually fine.

4 Doc values: columns for counting and sorting



Figure 3. Matching needs term → docs. Counting, sorting and scripting need doc → value, which is what doc values store.

An aggregation needs the reverse of the inverted index: for each matching doc, its brand and price. **Doc values** are an on-disk, column-oriented copy of each field's values, written at index time and read through the OS page cache. They are enabled by default for keyword, numeric, date, boolean and geo fields. Analysed `text` fields have no doc values: aggregating them needs `fielddata`, which builds the column on the JVM heap and is off by default for good reason. So facet on `keyword` or numeric fields (e.g. `brand.keyword`), never on the analysed name. Sorting by price uses the same columns.

5 The facet UX trap: post filters

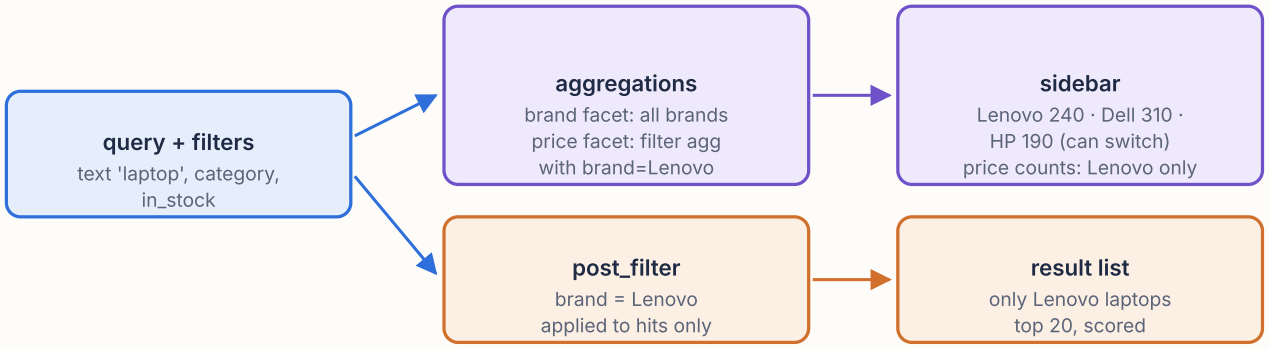


Figure 4. Multi-select facets: the brand filter goes in post_filter so the brand facet still shows other brands, while other facets respect it.

If the brand filter goes into the main query, the brand facet now shows only 'Lenovo 240', and the shopper can't see or add Dell. `post_filter` is applied *after* aggregations are computed, so it narrows the hits but not the facets. Other facets (price, screen) should still respect the chosen brand, so wrap them in a `filter` aggregation with the same brand condition. The general rule: each facet is computed with every selected filter **except its own**. Post filters are slower than normal filters (they can't narrow the work done by aggregations), so use them only for multi-select facets.

WORKED EXAMPLE: CATEGORY AND PRICE FACETS IN ONE ROUND TRIP

The request in Figure 2 carries the text query 'laptop', filters `category=laptops` and `in_stock=true`, a terms aggregation on `brand` and a range aggregation with buckets <500, 500–1000 and ≥1000. Each shard scores its hits, keeps its local top 20, and builds partial brand and price buckets from doc values; the coordinating node merges them (lesson 25). The response holds 20 results plus e.g. Lenovo 240 · Dell 310 · HP 190 and <$500: 1,900 · $500–1,000: 3,100 · ≥$1,000: 7,000, in tens of milliseconds. The two filters are cached after their first few uses, so the next shopper browsing laptops pays almost nothing for them. When she ticks Lenovo, brand moves into `post_filter` and the price aggregation gets a brand filter.

TRADE-OFFS

- **Exact vs fast counts**: bigger `shard_size` means more accurate terms counts and more work per shard.
- **Many facets** make every query heavier; compute rarely-used facets only when the user opens them.
- **High-cardinality facets** (seller ID across millions) are expensive; precompute or limit them.
- **Post filters** give natural multi-select UX but can't speed up aggregations.

COMMON MISTAKES

- Putting yes/no conditions in `must`: they get scored and are never cached.
- Faceting on an analysed text field ('lenovo thinkpad' split into words) or turning on fielddata to make it work.
- Using `now` without rounding in filters, so nothing is ever cached.
- Applying the selected brand to the main query, so the brand facet collapses to one option.

6 Key terms

Filter context: yes/no clause, no scoring, cacheable

Aggregation: computes buckets or metrics over matching docs

post_filter: filters hits after aggregations are computed

Bit set: one bit per doc per segment: matches or not

Doc values: columnar on-disk field values for aggs and sorting

shard_size: buckets each shard returns before merging

7 Interview questions

Q1. How are facet counts computed? BASIC

With aggregations over the documents matching the current query and filters, e.g. terms for brands and range for prices, read from column-oriented doc values.

Q2. Why use filters instead of queries for conditions? BASIC

Filters skip scoring and their results can be cached as per-segment bit sets, so they are faster, especially for repeated conditions.

Q3. Brand facet shows only the selected brand after the user clicks it. Fix? SCENARIO

Move the brand condition into post_filter so the brand aggregation ignores it, and add it as a filter aggregation around the other facets.

Q4. Facet counts on a 30-shard index don't add up exactly. Why? SCENARIO

terms aggregations take each shard's local top shard_size buckets, so a value outside some shards' top lists is under-counted; raise shard_size or accept the documented error bound.

Q5. A 'last 24 hours' filter is slow despite the filter cache. Why? SCENARIO

A filter using raw now differs on every request, so it is never cached; round it (now-24h/h) so the same filter repeats.

Q6. Why can't you aggregate on a text field by default? TRICKY

Text fields have no doc values; aggregating needs fielddata built on the heap from the inverted index, which is memory-hungry and would count analysed tokens, not the original values.

Q7. Do filters change the ranking? TRICKY

Not the scores of the remaining documents, but they change which documents are present; and a filter wrapped in a should clause or constant_score can contribute a fixed score if you ask it to.

Next: SD-P-006 · Indexing Pipelines backendroadmap.com